Performance Analysis of Parallel Mandelbrot Set Computation

Cansu Karagöz
ckarag00@estudiantes.unileon.es

March 30, 2026

Abstract

This report investigates the parallelization of the Mandelbrot set generation algorithm using OpenMP. The research addresses high-performance computing strategies, the challenge of load imbalance in fractal generation, the implementation of Static, Dynamic, and Guided scheduling, and the analysis of results which show that Dynamic scheduling achieves a super-linear speedup of 15.7x on 12 threads.

1 Introduction

The objective of this project is to parallelize the Mandelbrot set generation algorithm using OpenMP. The Mandelbrot set is computationally intensive as each pixel requires an iterative calculation. This project explores the performance impact of different thread counts and scheduling strategies to optimize rendering times.

2 Methodology

Three different versions of the code were evaluated:

- **Sequential:** The original single-threaded baseline.
- **Parallel Basic:** Initial parallelization using static scheduling.
- **Parallel Optimized:** Advanced parallelization using dynamic scheduling, reduction for global maximum calculation, and parallelized RGB color assignment.

3 Performance Comparison and Analysis

The execution times recorded for the different OpenMP scheduling strategies and thread counts are summarized in Table 1. The sequential baseline, measured at **619.7 seconds**, serves as the primary reference point for evaluating speedup and efficiency.

3.1 Impact of Thread Count and Scalability

As shown in the experimental data, increasing the number of threads (P) from 1 to 12 resulted in a significant reduction in wall-clock time across all parallel implementations.

- **Scalability Limits in Static Scheduling:** With `static` scheduling, performance improvements showed diminishing returns beyond 8 threads, with the execution time stagnating around 102.8 seconds at $P = 12$. This indicates that a simple uniform distribution of iterations is insufficient for the Mandelbrot set's irregular computational nature.
- **Dynamic vs. Static (Peak Performance):** The **Parallel Optimized** version utilizing `dynamic` scheduling achieved the peak performance of this study, reaching a minimum execution time of **39.29 seconds** on 12 threads. This configuration outperformed the static approach by approximately **2.6x**. The efficiency stems from the scheduler's ability to assign work to threads as they become idle, effectively balancing the load between points inside the set (high iteration) and those outside (quick escape).
- **Guided Scheduling:** While `guided` scheduling (70.92 s at 12 threads) performed better than the static baseline, it remained less efficient than the dynamic strategy. Although it reduces overhead by starting with larger chunks, it failed to provide the fine-grained load balancing required to handle the complex boundaries of the fractal efficiently.

Table 1: Execution Time (s) per Scheduling Strategy

Threads (P)	**Static (s)**	**Dynamic (s)**	**Guided (s)**
1 (Sequential)	619.7	N/A	N/A
2	250.5	197.6	309.4
4	251.1	100.1	140.1
8	136.4	59.29	85.16
12	102.8	**39.29**	70.92

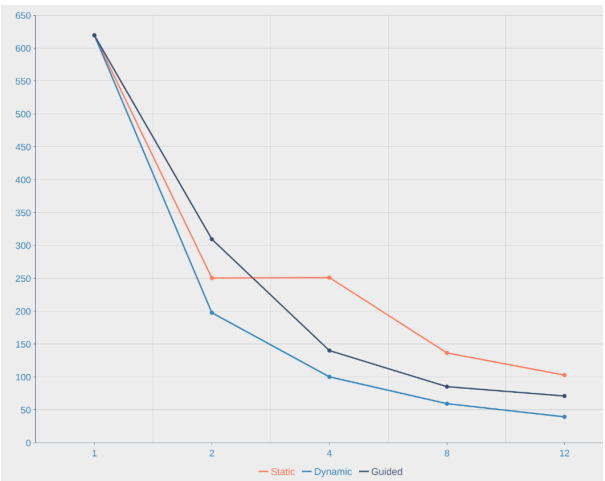

Figure 1: Execution Time (s) vs. Number of Threads for different OpenMP scheduling strategies.

4 Metrics and Discussion

For the best performing case (12 threads, Dynamic):

- **Speedup (S):** $S = \frac{T_{seq}}{T_{par}} = \frac{619.7}{39.29} \approx 15.77$
- **Efficiency (E):** $E = \frac{S}{P} = \frac{15.77}{12} \approx 1.31$ (131%)

The observed efficiency of **131%** confirms a **super-linear speedup**. This phenomenon is primarily attributed to the **Cache Effect**. By partitioning the large 10501×10501 grid into smaller chunks, the working set of each thread fits more effectively into the CPU's $L2$ and $L3$ caches. This drastically reduces high-latency access to main memory (RAM), allowing the parallel implementation to exceed the theoretical linear speedup limit.

5 Visual Output

The visual output of the optimized program was validated to ensure that parallelization did not affect the mathematical integrity of the fractal. Figure 2 illustrates the final render using the optimized RGB color mapping.

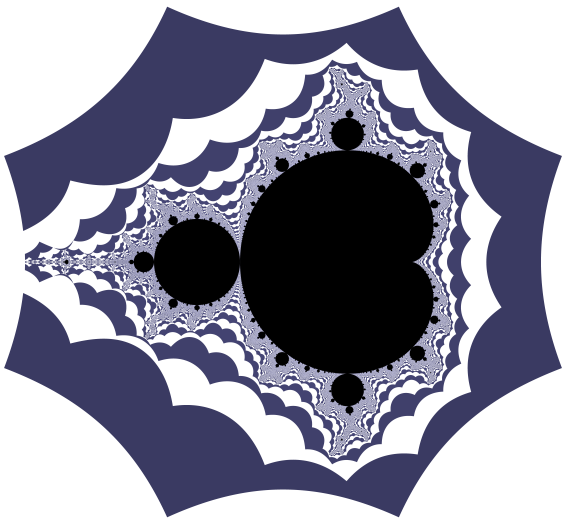

Figure 2: Rendered Mandelbrot Set (10501×10501 resolution).

6 Conclusion

The parallel implementation successfully reduced execution time from 619.7 s to 39.29 s. Using `#pragma omp parallel for reduction(max:c_max)` ensured correctness, while dynamic scheduling provided the best workload distribution for this specific fractal geometry.